Writing a Simple Vim Plugin for REPL Interaction

An alternative to Nvim-R and vim-slime for sending code to an R terminal

Ronald 'Ryy' G. Thomas

2025-05-20

Table of contents

Figure 1: A terminal split between a Vim editing buffer and a running R session with code being sent between them

A lightweight Vim plugin for sending R code to a terminal REPL.

1 Introduction

I did not really know how Vim's terminal API worked until I sat down and wrote a small plugin that sends code from an editing buffer to a running R session. The established tools for this task – Nvim-R and vim-slime – are feature-rich but complex. I wanted something minimal: select code in Vim, press a key, and see it execute in an R terminal pane.

The result is `rgt-R.vim`, a single-file plugin under 150 lines that handles line submission, visual selection, chunk navigation, and a handful of convenience mappings for common R inspection commands. It uses Vim's built-in `term_sendkeys()` and `term_list()` functions, requiring no external dependencies beyond Vim 8+ with terminal support.

This post presents the complete plugin code, explains each function, documents the key mappings, and discusses the experiments that shaped the design.

1.1 Motivations

- Nvim-R is powerful but introduces substantial complexity for users who only need code submission; I wanted a lightweight alternative.
- vim-slime uses tmux or screen as intermediaries, adding configuration overhead that I wanted to avoid for a simple R workflow.
- Understanding Vim's terminal API (`term_sendkeys`, `term_list`) seemed valuable for building custom development workflows beyond R.
- I teach R programming and wanted a plugin simple enough that students could read and understand the source code in a single sitting.
- I was curious whether a minimal plugin could cover 90% of my daily R interaction needs without the remaining 10% of features that add complexity.

1.2 Objectives

1. Build a Vim plugin that sends individual lines, visual selections, and R Markdown chunks to a running R terminal using Vim's built-in terminal API.
2. Document each VimScript function with its purpose, mechanism, and interaction with the terminal buffer.
3. Explain the autocommand group that restricts mappings to R, Rmd, and Qmd filetypes.
4. Present the experiments that explored edge cases in visual selection submission and Quarto rendering.

I am documenting my learning process here. If you spot errors or have better approaches, please let me know.

Understanding Vim's terminal API opens the door to custom development workflows.

2 Prerequisites and Setup

To use this plugin, you will need:

- Vim 8.0 or later with `+terminal` feature (check with `:echo has('terminal')`)
- R installed and accessible from the command line
- The plugin file placed in your Vim plugin directory (e.g., `~/.vim/plugin/rgt-R.vim`)

Background assumed: Basic Vim fluency (normal mode, visual mode, command mode) and familiarity with R. No prior VimScript experience is required; this post explains each function.

To verify terminal support:

```
:echo has('terminal')
```

If this returns `1`, your Vim build supports the terminal API. If it returns `0`, you will need to install a Vim build with terminal support (e.g., `brew install vim` on macOS).

3 What is a Vim REPL Plugin?

A REPL (Read-Eval-Print Loop) plugin connects a text editor to a running interpreter. You write code in the editor, press a key combination, and the code is sent to the interpreter for execution. The interpreter prints its output, and you continue editing.

Vim's terminal API provides the plumbing for this connection. The function `term_list()` returns a list of open terminal buffers, and `term_sendkeys()` sends keystrokes to a specific terminal. By combining these with Vim's visual selection and search capabilities, a small plugin can provide a complete code-submission workflow.

Think of it as a pipe between your editor and your R session. The plugin handles the mechanics of extracting selected text and pushing it through the pipe; R handles the evaluation and output.

4 Getting Started

4.1 Launching an R Terminal

The plugin includes a mapping to open an R terminal in a vertical split. Press `<localleader>r` in normal mode:

```
nnoremap <silent> <localleader>r
  \ :vert term R --no-save<CR><c-w>:wincmd p<CR>
```

This opens R in a vertical terminal split and returns focus to the editing buffer. The `--no-save` flag prevents R from prompting to save the workspace on exit.

5 The Complete Plugin

5.1 Core Submission Functions

5.1.1 SubmitLine: Send the Current Line

```
function! SubmitLine()
  :let @c = getline(".") . "\n"
  :call term_sendkeys(term_list()[0], @c)
endfunction
```

`SubmitLine` copies the current line into register `c`, appends a newline character, and sends it to the first terminal in `term_list()`. The newline triggers R to evaluate the expression.

Mapping: Press `<CR>` (Enter) in normal mode on any R, Rmd, or Qmd file to submit the current line.

5.1.2 GetVisualSelection: Extract Selected Text

```
function! GetVisualSelection(mode)
  let [line_start, column_start] =
    \ getpos("'<")[1:2]
  let [line_end, column_end] =
    \ getpos("'>")[1:2]
  let lines = getline(line_start, line_end)
  if a:mode ==# 'v'
    let lines[-1] = lines[-1][
      \ : column_end -
      \ (&selection == 'inclusive' ? 1 : 2)]
    let lines[0] = lines[0][column_start - 1:]
  elseif a:mode ==# 'V'
  else
    return ''
  endif
  return join(lines, "\n")
endfunction
```

`GetVisualSelection` handles both character-wise (`v`) and line-wise (`V`) visual selections. For character-wise selection, it trims to the exact column range. For line-wise selection, it returns the full lines without modification. The function returns the selected text as a single string with newlines preserved.

5.1.3 Sel and Submit: Two-Stage Submission

```
function! Sel()
  :let @c = GetVisualSelection(
    \ visualmode()) . "\n"
  :call writefile(
    \ getreg('c', 1, 1), "source_visual")
endfunction

function! Submit()
  :let y = "source('source_visual',echo=T)"
    \ . "\n"
  :call term_sendkeys(term_list()[0], y)
endfunction
```

Visual selection submission uses a two-stage process: `Sel` writes the selected text to a temporary file (`source_visual`), and `Submit` tells R to source that file with echo enabled. This approach avoids the complexity of sending multi-line text directly through `term_sendkeys`, which can cause timing issues with long selections.

Mapping: Select code visually and press `<CR>` to submit the selection.

5.1.4 Brk: Interrupt R

```
function! Brk()
  :call term_sendkeys(
    \ term_list()[0], "\<c-c>")
endfunction
```

`Brk` sends Ctrl-C to the R terminal, interrupting any running computation. Mapped to `<local-leader>c`.

5.2 Chunk Navigation Functions

5.2.1 SelectChunk: Select an R Markdown Chunk

```
function! SelectChunk()
  :execute
    \ "normal! ?```{\<cr>jV/```\<cr>k"
endfunction
```

`SelectChunk` searches backward for a code fence opening (```{), moves down one line, enters visual line mode, searches forward for the closing fence, and moves up one line. The result is a visual selection of the chunk contents, excluding the fences.

5.2.2 MoveNextChunk and MovePrevChunk

```
function! MoveNextChunk()
  :execute "normal! /```{\<CR>j"
  :noh
endfunction

function! MovePrevChunk()
  :execute "normal! 2?```{\<CR>j"
  :noh
endfunction
```

These navigation functions move the cursor to the first line of the next or previous code chunk. The `:noh` command clears the search highlighting. `MovePrevChunk` searches backward twice because the first match is the current chunk's opening fence.

Mappings: `<localleader>j` for next chunk, `<localleader>k` for previous chunk.

5.3 Convenience Functions

5.3.1 Raction: Quick R Inspection

```
function! Raction(action)
  :let @c = expand("<cword>")
  :let @d = a:action . "(" . @c . ")\n"
  :call term_sendkeys(term_list()[0], @d)
endfunction
```

`Raction` takes an R function name as its argument, wraps the word under the cursor as its argument, and sends the call to R. This enables quick inspection mappings:

Mapping	R Command	Purpose
`<localleader>d`	`dim(x)`	Dimensions
`<localleader>h`	`head(x)`	First rows
`<localleader>s`	`str(x)`	Structure
`<localleader>p`	`print(x)`	Print
`<localleader>n`	`names(x)`	Column names
`<localleader>f`	`length(x)`	Length

5.3.2 SubmitEmbed and Rd: Inline Output

```
function! SubmitEmbed()
  :let y = "sink('temp.txt'); " .
    \ "source('source_visual',echo=T); " .
    \ "sink()" . "\n"
```